OSTIM TECHNICAL UNIVERSITY

FACULTY OF ENGINEERING
ELECTRICAL AND ELECTRONICS ENGINEERING

EEE308 – MICROPROCESSORS TERM PROJECTS REPORT

ARDUINO UNO BASED – SMART PARKING SYSTEM

Nursena GÜLER – 220202016

Spring 2025-2026

1 Introduction

The number of vehicles on the road is growing every year. Because of this, finding a parking space in crowded places is becoming harder and harder. Drivers often waste a lot of time driving around looking for an empty spot, especially in indoor parking lots where it is not easy to see which spaces are taken.

To solve this problem, this project presents an Arduino Uno based Smart Parking System. The system uses sensors to detect when a vehicle arrives at the entrance and checks all parking spaces in real time. It then shows the driver which spaces are empty and directs them to the nearest available spot. The entrance gate opens automatically when a vehicle is detected, and closes again after the vehicle enters.

The whole system was first designed and tested in a simulation environment (Proteus), and then built with real hardware components on a physical prototype. This allowed a direct comparison between the ideal simulation world and real-world behavior.

This report covers everything about the project: the components used, the circuit design, the software logic, the differences between the simulation and the real system, and the results.

2 Project Objectives

The main goals of this project are:

- To automate the parking lot entrance so drivers do not need to stop and wait for a person to open the gate.
- To detect whether each parking space is empty or occupied in real time using infrared (IR) sensors.
- To show the driver how many spaces are available and which ones they are.
- To guide the driver to the nearest empty parking space.
- To control the entrance gate automatically using a servo motor — opening when a vehicle arrives, and closing after it enters.
- To display all information clearly on a 16x2 LCD screen.
- To keep the system full parking lot scenario in mind: if all spaces are taken, the gate stays closed and the driver is informed.

3 System General Structure

The system is built in a modular way. Each module has a clear job, and they all work together under the control of the Arduino Uno. The five main modules are:

1. Brain (Arduino Uno)
2. Information System (LCD screen)

3. Vehicle Detection System (Ultrasonic sensor)
4. Access Control System (Servo motor)
5. Parking Area Control System (IR sensors)

3.1 Microcontroller – Arduino Uno

The Arduino Uno is the brain of the system like this serves as the central control unit of the system. It coordinates all operations by receiving inputs from the sensors, processing this data, and controlling the output components.

All modules are connected to the Arduino, which ensures synchronized operation of the entire system.

3.2 Display Module – I2C LCD

The display module is responsible for providing real-time feedback to the user. It acts as a communication interface between the system and the driver.

The LCD screen displays system status messages such as vehicle detection, available parking spaces, parking suggestions, and door operations. It also displays waiting information when no vehicle is detected.

3.3 Vehicle Detection Module – Ultrasonic Sensor

The vehicle detection module is used to identify incoming vehicles at the entrance of the parking area.

This module continuously monitors the distance in front of the system. When a vehicle enters a predefined detection range, the system recognizes the presence of a car and initiates the parking process.

3.4 Parking Space Detection Module – IR Sensors

The parking space detection module monitors the occupancy status of each parking slot.

Each parking space is equipped with an IR sensor that determines whether the space is occupied or empty. The system evaluates all sensor inputs to determine available parking locations.

3.5 Gate Control Module – Servo Motor

The gate control module manages the opening and closing of the parking lot entrance.

Depending on the system's decision, the servo motor rotates 90 degrees to allow or block vehicle entry. This ensures controlled and automated access to the parking area.

4 Components Used

All hardware components used in this project were selected with the system's reliability, understandability, and practicality in mind. Each component performs a specific task within the system and works seamlessly with other components.

The table below summarizes the main components used, their roles in the system, and the reasons for their selection:

Components	Its Role Within the System	Reason for Selection
Arduino Uno	Central control unit	Easy programming, extensive library support, sufficient input/output pins.
Ultrasonic Sensor (HC-SR04)	Vehicle detection	Non-contact measurement, fast and reliable distance calculation.
IR Sensors (4)	Parking occupancy assessment	Simple digital output, fast response.
Servo Motor (SG90)	Barrier control	Precise angle control, low cost.
16x2 I2C LCD	User information	Low pin usage, stable communication.
Breadboard & Jumper Wire	Circuit connections	Rapid prototyping without soldering.

4.1 Arduino Uno

The Arduino Uno acts as the brain of the system. It is the main control unit that collects and processes data from all sensors and decides how the system should behave accordingly.

This board, equipped with an ATmega328P microcontroller, offers both digital and analog input/output support. It facilitated the easy integration of different components in the project.

In this project, the Arduino detects vehicle presence, analyzes the status of parking spaces, determines the most suitable parking space, informs the user, and controls the entrance gate.

Thanks to these features, the Arduino Uno provides sufficient performance for the system.

4.2 Ultrasonic Sensor (HC-SR04)

An ultrasonic sensor is used in the system to detect vehicle entry. Thanks to its contactless measurement capability, this sensor can operate reliably for a long time.
The sensor sends high-frequency sound waves and measures the time it takes for these waves to hit an object and return. This measured time is used in distance calculation.
The distance calculation is based on the following formula:
Distance = (Time × Speed of Sound) / 2

This method allows the system to detect an approaching vehicle based on a specific distance threshold.

4.3 IR Sensors

IR sensors are used to determine whether each parking space in the parking lot is occupied or empty. The sensors emit infrared light and determine whether the light is reflected back. If the light is reflected, there is a vehicle in the parking space. If the light is not reflected, the parking space is empty. The sensors produce a digital output. HIGH represents an empty parking space, and LOW represents an occupied parking space. The sensitivity can be adjusted with the potentiometer. It provides a fast and instantaneous response. It produces a simple digital signal.

4.4 Servo Motor (SG90)

A servo motor was used to control the parking garage entrance gate. This motor has the ability to rotate precisely to specific angles. Servo motors operate with PWM (Pulse Width Modulation) signals. The angle of the motor is controlled by changing the pulse width of the PWM signal. Arduino's Servo library automatically manages this process and generates a precise signal using a timer in the background. In the real system, the servo motor angle was adjusted by physically testing it:

20° → Gate closed (parallel to the ground)
110° → Gate open

These values differ from the simulation (0°/90°) and were determined according to the mechanical structure of the real system.

4.5 I2C LCD (16x2 Screen)

The LCD screen is the unit that provides interaction between the user and the system. All information about the system status is conveyed to the user via this screen. Vehicle detection information, number of empty parking spaces, empty parking areas, suggested parking space, door status.

In this project, the LCD screen is connected using the I2C communication protocol. I2C is a serial communication protocol that works over two wires. SDA is the data line, SCL is the timing (clock) line.

Data transmission occurs synchronously with the clock signal. The Arduino (master) sends data to the LCD (slave), and the LCD reads this data according to the clock signal.

It requires very few connections compared to a normal LCD; only 2 lines are used. It reduces cable clutter. It provides stable communication.

The LCD address differed between the simulation and the real system:

Simulation (Proteus): 0x20

Real system: 0x27

This difference is due to the address pins on the I2C converter used.

5 Circuit Design And Pin Connections

All components used in this project are connected to the Arduino Uno microcontroller via specific pins. Both hardware compatibility and ease of use were considered during the circuit design.

One of the most important rules for the correct and stable operation of the system is that all modules share a common ground (GND) line. Without a common GND connection, the system can experience inaccurate measurements, unstable behavior, and communication problems.

5.1 Pin Connection Table

Arduino Pin	Connected Component	Signal Type
D2	IR Sensor 1 (Parking 1)	Digital Input
D3	IR Sensor 2 (Parking 2)	Digital Input
D4	IR Sensor 3 (Parking 3)	Digital Input
D5	IR Sensör 4 (Parking 4)	Digital Input
D6	Ultrasonic ECHO	Digital Input (duration measurement)
D7	Ultrasonic TRIG	Digital Output
D9	Servo Motor	PWM Output
A4 (SDA)	I2C LCD Data Line	Series Data (I2C)

Arduino Pin	Connected Component	Signal Type
A5 (SCL)	I2C LCD Clock Line	Clock Signal (I2C)

5.2 Decisions Made in Circuit Design

During circuit design, the appropriate pin selection for each component was made by examining the data sheets and their operating principles. These selections are of great importance for the stable and accurate operation of the system.

5.2.1 Ultrasonic Sensor Connection

The ultrasonic sensor operates with two different signal lines:

TRIG pin → sends a signal to the sensor (output)
ECHO pin → reads the returned signal (input)

Therefore:

The TRIG (D7) pin is set as OUTPUT
The ECHO (D6) pin is used as INPUT

Although the ECHO pin supports PWM, the PWM feature is not used in this project. This pin is only used as a digital input to measure the duration that the returned signal remains at the HIGH level. Therefore, PWM support is not required, and any suitable digital input pin is sufficient for this task. The choice of the D6 pin was made to keep the pin layout neat and understandable.

5.2.2 Servo Motor Connection

A PWM signal is required to control the servo motor. Thanks to the PWM signal, the servo motor can be rotated precisely to specific angles. Therefore, the servo motor is connected to the D9 pin, which supports PWM. The Arduino Uno has 6 PWM pins: D3, D5, D6, D9, D10, and D11. However, for my project, the D9 pin was chosen based on pin suitability. It is controlled using the Arduino Servo library. The Servo library uses hardware timers in the background to generate the PWM signal precisely. This ensures that the servo motor's angle is adjusted accurately and stably.

5.2.3 I2C LCD Connection

The LCD screen is connected to the Arduino via the I2C (Inter-Integrated Circuit) serial communication protocol. This protocol allows multiple devices to share the same data bus using two lines:

SDA (Serial Data) → data line
SCL (Serial Clock) → timing (clock) line

These lines are fixed in hardware on the Arduino Uno:

A4 → SDA
A5 → SCL

These pins are connected to the ATmega328P microcontroller's internal TWI (Two-Wire Interface) module. The TWI module generates START/STOP conditions, creates the clock signal, transmits data bits sequentially, and manages the ACK/NACK (acknowledgment) mechanism. In this way, data transmission is performed synchronously and reliably in hardware without relying on software. These pins are connected to the microcontroller's internal I2C (TWI) module and cannot be changed via software. This ensures more reliable and synchronous data transmission via hardware.

5.2.4 IR Sensor Connections

IR sensors were used to determine the occupied/empty status of parking spaces. The sensors were connected to the Arduino as follows:

IR1 → D2
IR2 → D3
IR3 → D4
IR4 → D5

IR sensor modules contain an IR LED (transmitter) and a receiver (photodiode/phototransistor). The transmitter emits infrared light; if an object is present, this light is reflected back and detected by the receiver.

If there is a reflection (tool present) → LOW (0)
If there is no reflection (empty) → HIGH (1)

The sensor output is digital, as seen, and a digital pin is used.

5.2.5 Ground and VCC Connection

During the actual circuit setup, a common GND line was created to ensure all components function properly. The Arduino Uno's GND pin was connected to the breadboard, and all sensors and modules were grounded via this line. Similarly, the 5V power supply was distributed from the breadboard. Furthermore, to reduce interference and ensure more stable operation, the VCC and GND wires were twisted together. This method aims to reduce noise, especially in sensitive sensors and communication lines, contributing to a more stable system operation.

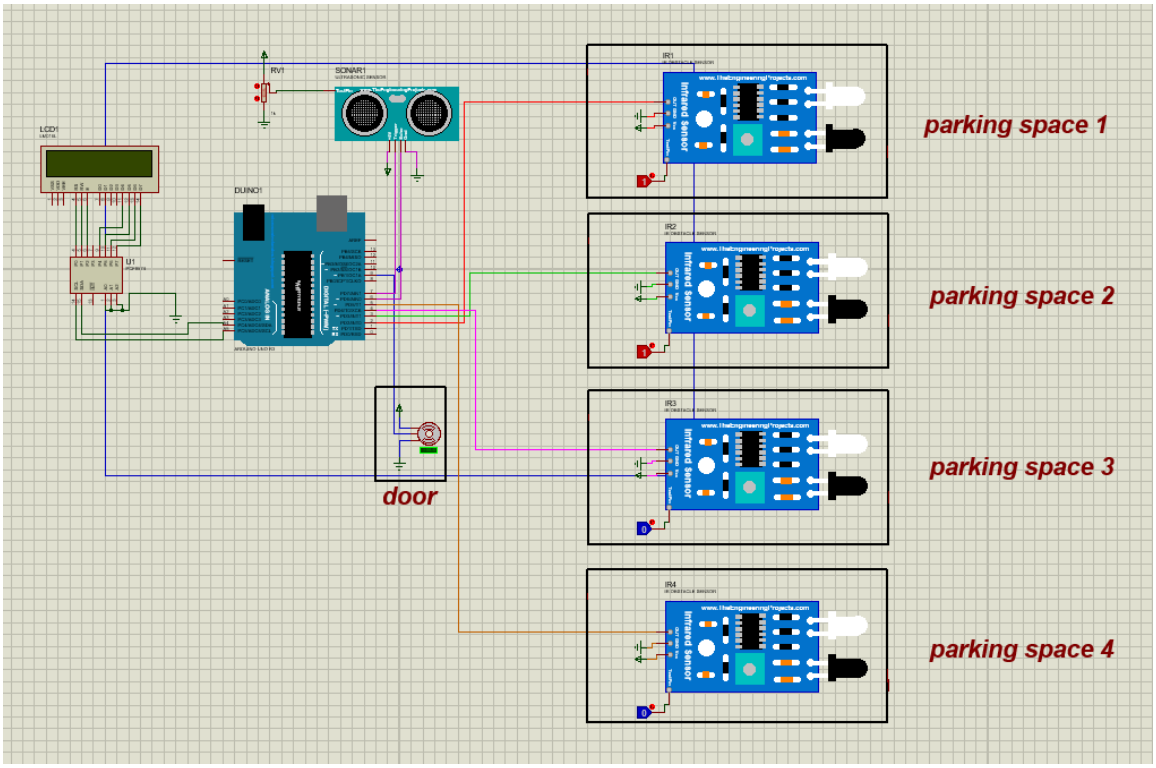

Figure 1: Proteus Circuit Diagram of the Smart Parking System

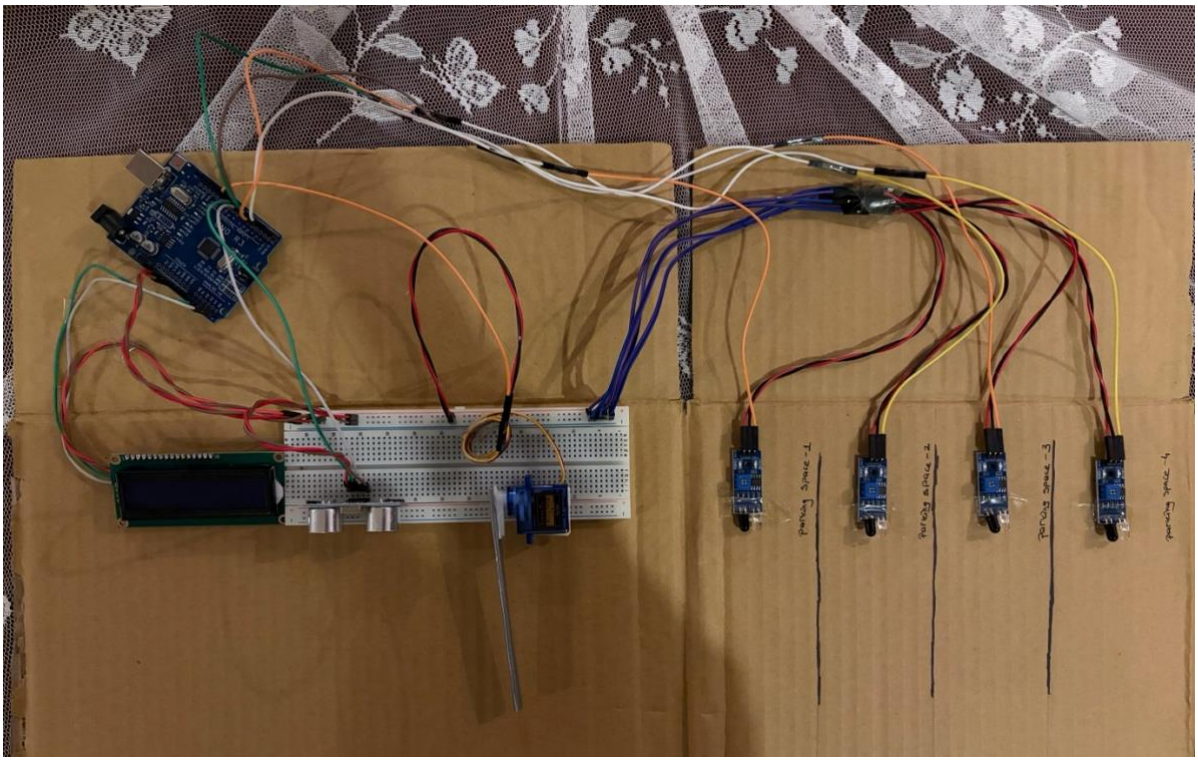

Figure 2: Real Circuit Diagram of the Smart Parking System

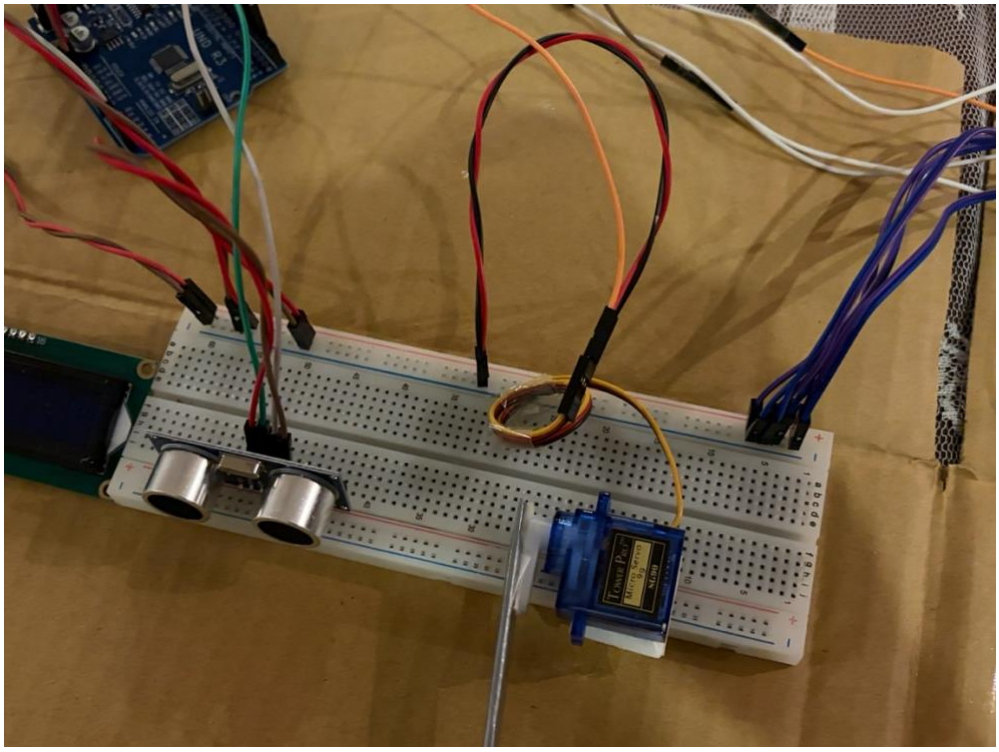

Figure 3: Real Circuit Diagram of the Smart Parking System for Breadboard

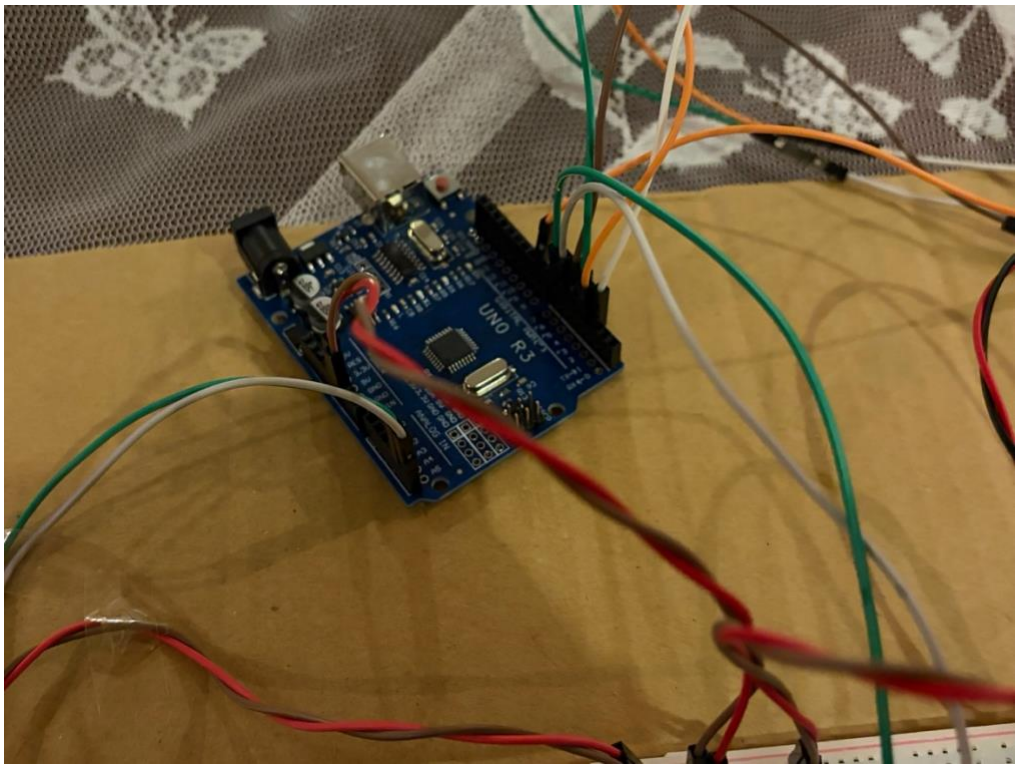

Figure 4: Real Circuit Diagram of the Smart Parking System for Arduino Uno

Although the circuit connections in the real system may seem complex at first glance, all connections were implemented exactly as designed in the Proteus simulation environment. In the physical installation, only some jumper cables were extended to give the parking areas a more realistic appearance. In addition, to reduce interference that may occur in the system and to ensure more stable operation, the VCC and GND lines were twisted together. These arrangements did not change the general working principle of the circuit, they only made it easier to adapt to the physical application.

6 SOFTWARE DESIGN

The software used in this project was developed using C/C++ in the Arduino IDE environment. Two different code versions were prepared for the system:

- Code written for Proteus simulation
- Code used for the actual hardware

Both codes share the same basic algorithm, but some parameters have been modified to adapt to the physical behavior of the system.

6.1 Libraries Used

The following libraries were used in the project:

- Wire.h →Provide I2C-based communication with Arduino
- LiquidCrystal_I2C.h → Allows easy control of the LCD screen
- Servo.h → Enables control of the servo motor with a PWM signal

6.2 Code Structure and Functions

The code is divided into auxiliary functions to improve readability and create a modular structure. Each function performs a specific task.

Function	Task
measureDistance()	It measures distance using an ultrasonic sensor.
isEmpty(irPin)	It checks if the parking space is empty.
countEmptySpaces()	Calculates the number of available parking spaces.
emptySpacesText()	Lists available parking spaces as text.

Function	Task
nearestEmptySpace()	Identifies the nearest available parking space.
showMessage()	Clears the LCD screen and displays a message.

This structure has made the code more organized, readable, and extensible.

6.3 Main Loop Operation Logic

The program's main control structure runs continuously within the loop() function. The system repeats the following steps:

1. The distance at the entrance is continuously measured using an ultrasonic sensor.
2. The system activates when the vehicle falls below a certain distance (≤ 12 cm).
3. The occupancy status of parking spaces is read from IR sensors.
4. The number of empty spaces is calculated.
5. If there are no empty spaces, the message "PARKING FULL" is displayed on the LCD, and the gate does not open.
6. If there are empty spaces:
- The number of empty spaces is displayed.
- Empty spaces are listed.
- The nearest parking space is suggested.
- The servo motor opens the gate.
7. The gate is closed after a certain period of time.
8. When the vehicle moves away (≥ 20 cm), the system resets itself.
9. The system enters standby mode and is ready for a new vehicle.

6.4 Software Differences Between Simulation and Real System

Some significant differences were observed between the simulation environment and the actual hardware, and adjustments were made to the code accordingly.

6.4.1 LCD Address Difference

Proteus: 0x20
Actual system: 0x27

This difference stems from the hardware address settings of the I2C module used.

6.4.2 Servo Motor Angle Difference

- Simulation:
  Off → 0°
  On → 90°

- Real System:
  Off → 20°
  On → 110°

This variation is the result of calibration based on the physical placement and mechanical angles of the servo motor.

6.4.3 Delay Times

The delay times used in the real system have been increased compared to the simulation. Sensor response time. Servo motor physical movement time. User's ability to read messages

For example:
- Message times were kept short in the simulation
- Longer times were used in the real system (1500 ms, etc.)

6.4.4 System Behavioral Stability

While components operate under ideal conditions in the simulation environment, in the real system; noise, sensor sensitivity, physical delays.

These factors affect system behavior. Therefore, the software has been optimized to suit the real system.

6.5 General Assessment

Thanks to its modular structure and functional organization, the developed software works successfully in both simulation and real systems. Analyzing the differences between simulation and real-world applications and adapting the software accordingly is a significant engineering achievement of the project.

7 Embedded Software Implementation

This section describes the embedded software developed for implementing a smart parking system. The system is programmed using the Arduino platform. The software performs tasks such as reading data from sensors, operating the decision-making mechanism, and controlling output units. Below is the code used to operate the Arduino. Explanatory comments have been added where necessary to make the code more understandable and to help the reader grasp the system more easily.

```
#include <Wire.h>                 // Used for I2C communication
#include <LiquidCrystal_I2C.h>    // Used to control the I2C LCD
#include <Servo.h>                // Used to control the servo motor

// LCD address is 0x27, display size is 16x2
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Servo object representing the gate
Servo gate;

// Ultrasonic sensor pins
const int trigPin = 7;   // TRIG: sends ultrasonic signal
const int echoPin = 6;   // ECHO: receives signal and measures time

// Servo motor control pin
const int servoPin = 9;  // Connected to PWM pin D9

// IR parking sensors
const int ir1 = 2;   // Parking space 1
const int ir2 = 3;   // Parking space 2
const int ir3 = 4;   // Parking space 3
const int ir4 = 5;   // Parking space 4

// Servo angles
const int gateClosed = 20;   // Gate closed position
const int gateOpen = 110;    // Gate open position

// Distance thresholds
const float carDetectDistance = 12.0;  // Detect car if closer than 12 cm
const float carClearDistance  = 20.0;  // Reset system if farther than 20 cm

// Prevents processing the same car multiple times
bool carProcessed = false;

// Checks if a parking space is empty
// HIGH = empty, LOW = occupied
bool isEmpty(int irPin) {
  return digitalRead(irPin) == HIGH;
}
```

```
// Measures distance using ultrasonic sensor
float measureDistance() {
  // Ensure TRIG starts LOW
  digitalWrite(trigPin, LOW);
  delayMicroseconds(5);

  // Send 10 µs pulse to trigger measurement
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Measure how long ECHO stays HIGH (timeout = 30000 µs)
  long duration = pulseIn(echoPin, HIGH, 30000);

  // If no signal is received, return a large distance
  if (duration == 0) {
    return 999.0;
  }

  // Convert time to distance (divide by 2 for round trip)
  float distance = duration * 0.0343 / 2.0;
  return distance;
}

// Counts how many parking spaces are empty
int countEmptySpaces() {
  int count = 0;

  if (isEmpty(ir1)) count++;
  if (isEmpty(ir2)) count++;
  if (isEmpty(ir3)) count++;
  if (isEmpty(ir4)) count++;

  return count;
}

// Creates a string listing empty parking spaces
String emptySpacesText() {
  String text = "";

  if (isEmpty(ir1)) text += "1 ";
  if (isEmpty(ir2)) text += "2 ";
  if (isEmpty(ir3)) text += "3 ";
  if (isEmpty(ir4)) text += "4 ";

  // If no spaces are empty
  if (text == "") text = "None";

  return text;
}
```

```
// Finds the nearest available parking space
int nearestEmptySpace() {
  if (isEmpty(ir1)) return 1;
  if (isEmpty(ir2)) return 2;
  if (isEmpty(ir3)) return 3;
  if (isEmpty(ir4)) return 4;

  // Return 0 if no space is available
  return 0;
}

// Displays two lines on the LCD with a delay
void showMessage(String line1, String line2, int waitTime) {
  lcd.clear();           // Clear LCD
  lcd.setCursor(0, 0);   // First row
  lcd.print(line1);

  lcd.setCursor(0, 1);   // Second row
  lcd.print(line2);

  delay(waitTime);       // Wait for readability
}

void setup() {
  // Initialize LCD and turn on backlight
  lcd.init();
  lcd.backlight();

  // Set ultrasonic pins
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  // Set IR sensor pins
  pinMode(ir1, INPUT);
  pinMode(ir2, INPUT);
  pinMode(ir3, INPUT);
  pinMode(ir4, INPUT);

  // Attach servo and set initial position (closed)
  gate.attach(servoPin);
  gate.write(gateClosed);

  // Show startup message
  showMessage("Smart Parking", "System Ready", 1500);
}

void loop() {
  // Continuously measure distance
  float distance = measureDistance();

  // If a car is detected and not processed yet
  if (!carProcessed && distance <= carDetectDistance) {
    carProcessed = true;
```

```
    // Get parking information
    int emptyCount = countEmptySpaces();
    String emptyText = emptySpacesText();
    int nearestPark = nearestEmptySpace();

    // Show welcome message
    showMessage("Welcome", "Car detected", 1500);

    // If parking is full
    if (emptyCount == 0) {
      showMessage("PARKING FULL", "Gate closed", 2000);
      gate.write(gateClosed);
    }
    else {
      // Show available spaces
      showMessage("Empty spaces:", String(emptyCount), 1500);
      showMessage("Available:", emptyText, 2000);
      showMessage("Go to:", String(nearestPark), 2000);

      // Open gate
      showMessage("Gate opening", "Please move", 1500);
      gate.write(gateOpen);

      delay(2500);  // Wait for car to pass

      // Close gate
      showMessage("Gate closing", "Please wait", 1500);
      gate.write(gateClosed);

      delay(1500);
    }

    lcd.clear();
  }

  // Reset system when car leaves
  if (carProcessed && distance >= carClearDistance) {
    carProcessed = false;
    showMessage("Ready for", "next car", 1500);
  }

  // Standby screen when no car is present
  if (!carProcessed) {
    lcd.setCursor(0, 0);
    lcd.print("Waiting for car");

    lcd.setCursor(0, 1);
    lcd.print("Distance:");
    lcd.print(distance, 1);
    lcd.print("cm   ");
  }

  // Small delay to stabilize loop
  delay(300);
}
```

8 Conclusion

This project successfully demonstrated how a microcontroller like the Arduino Uno can be used to build a practical, real-world system. The Smart Parking System works as expected: it detects vehicles at the entrance, checks all parking spaces, guides the driver, and automatically controls the gate.

Building both a simulation (Proteus) and a physical prototype provided a very valuable experience. It showed that the real hardware has extra challenges that the simulation hides – such as sensor calibration, timing adjustments, I2C address differences, and the importance of the shared ground connection.

The things learned from this project are:

• Simulation is a great tool for testing logic and connections before building. However, it is not a replacement for real-world testing.

• Small differences in code settings (such as I2C address or servo angle) can make a big difference in whether the real hardware works or not.

• Modular design makes testing and correcting a system much easier. Each module (sensors, gate, display) can be tested independently.

• Actual sensor behavior is environment-dependent. IR sensors must be calibrated. Ultrasonic sensors can be affected by noise or reflections.

8.1 Future Developments

If this project is further developed, various features can be added:

• Mobile Application Integration: A smartphone application can remotely display the parking status, so drivers know if there are empty spaces before they arrive.

• IoT Connectivity: By adding a Wi-Fi module (ESP32), the system can send real-time data to a web server or cloud platform.

• Camera-Based Detection: Instead of IR sensors, cameras and image processing can be used for more reliable and accurate location detection.

• License Plate Recognition: A camera at the entrance can read license plates for security and access control.

• Multiple Entry/Exit Points: The system can be expanded to handle multiple entry or exit gates.

In conclusion, this project is a working proof of concept for a smart parking system. All the initially stated goals have been achieved, and the project provides a solid foundation for more advanced work in the future.